BEDAH KODE & PANDUAN TANYA-JAWAB TEKNIS NOTEBOOK (.IPYNB)

Mini Project PMML: 20 Pertanyaan Kritis Terkait Implementasi Kode Python/TensorFlow Beserta Jawaban Konseptual, Matematis, dan Praktis

File Sumber:	pmml_mini_project.ipynb & src/	**Dosen Pengampu:**	Afiahayati, S.Kom., M.Cs., Ph.D
Tujuan Dokumen:	Persiapan Sidang / Sesi Tanya-Jawab Teknis (Q&A;)	**Kelompok:**	Frans, Fikry, Rafel, Sidikara

BAGIAN 1: PREPROCESSING DATA & REKAYASA FITUR

[Q1] TARGET INVERSION: Mengapa Anda menulis kode df['target'] = (df['target'] == 0).astype(int)? Mengapa tidak menggunakan label bawaan scikit-learn?

```
Potongan Kode Terkait:
df['target'] = (df['target'] == 0).astype(int)
df['diagnosis'] = df['target'].map({1: 'Malignant (Ganas)', 0: 'Benign (Jinak)'})
```

Jawaban Komprehensif:
Secara default pada modul sklearn.datasets.load_breast_cancer, kelas 0 dilabeli sebagai *malignant* (ganas) dan kelas 1 sebagai *benign* (jinak). Dalam terminologi klinis dan metrik evaluasi medis standar (seperti Sensitivity/Recall), **Positive Class (label 1)** wajib merepresentasikan penyakit/kondisi keganasan yang ingin dideteksi. Dengan membalik label agar 1 = Malignant dan 0 = Benign, metrik **Recall** secara otomatis mengukur sensitivitas model dalam mendeteksi tumor kanker ganas. Jika tidak dibalik, Recall akan mengukur seberapa baik model mendeteksi tumor jinak, yang secara klinis kurang tepat.

■ ***Poin Kunci Jawaban:*** *Menyesuaikan Positive Class (1) agar merepresentasikan tumor kanker ganas sehingga Recall tepat mengukur Sensitivitas Deteksi Kanker.*

[Q2] PENCEGAHAN DATA LEAKAGE: Mengapa pada StandardScaler Anda memanggil fit_transform pada X_train_val, tetapi hanya memanggil transform pada X_test?

```
Potongan Kode Terkait:
scaler = StandardScaler()
X_train_val_scaled = scaler.fit_transform(X_train_val)
X_test_scaled = scaler.transform(X_test)
```

Jawaban Komprehensif:
Ini adalah prinsip paling mendasar untuk mencegah **Data Leakage** (kebocoran informasi). Metode fit_transform menghitung parameter statistik (rata-rata μ dan standar deviasi σ) dari data latih, lalu menstandardisasi data latih tersebut: $z = (x - \mu) / \sigma$. Jika kita memanggil fit pada data uji atau pada keseluruhan dataset di awal, informasi distribusi data uji akan bocor ke dalam proses pelatihan. Oleh karena itu, data uji (test set) hanya boleh di-*transform* menggunakan μ dan σ yang murni dipelajari dari data training, merefleksikan skenario pasien baru di rumah sakit.

■ ***Poin Kunci Jawaban:*** *Mencegah kebocoran parameter statistik (mean dan varians) dari data uji ke model (Anti-Data Leakage).*

[Q3] DATA SPLITTING: Apa fungsi parameter stratify=y pada fungsi train_test_split dan StratifiedKFold?

```
Potongan Kode Terkait:
X_train_val, X_test, y_train_val, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

Jawaban Komprehensif:
Dataset Breast Cancer Wisconsin memiliki distribusi kelas yang tidak seimbang secara sempurna (imbalance): 62.7% Jinak (357) dan 37.3% Ganas (212). Jika kita melakukan pembagian acak biasa tanpa stratifikasi, ada kemungkinan partisi data uji atau salah satu lipatan (fold) memiliki proporsi tumor ganas yang jauh lebih sedikit atau lebih banyak. Parameter stratify=y memaksa algoritma pembagian untuk mengunci rasio 62.7% : 37.3% secara presisi di data training maupun data testing, serta di kelima fold validasi silang.

■ ***Poin Kunci Jawaban:*** *Menjaga rasio perbandingan kelas mayoritas dan minoritas tetap identik di seluruh lipatan data.*

[Q4] PILIHAN SKALASI: Mengapa Anda memilih StandardScaler (Z-score) dibandingkan MinMaxScaler?

Potongan Kode Terkait:
```
from sklearn.preprocessing import StandardScaler
```

Jawaban Komprehensif:
30 fitur klinis pada dataset ini (seperti area worst, perimeter, dll.) memiliki rentang nilai yang sangat berbeda (misal: area bernilai ribuan, sedangkan fractal dimension bernilai desimal 0.05). Selain itu, distribusi fitur morfologi sel FNA mendekati distribusi Gaussian (normal) dengan kemungkinan adanya nilai ekstrem (*outliers*). StandardScaler mentransformasikan fitur agar memiliki $\mu=0$ dan $\sigma=1$, yang sangat ramah terhadap fungsi aktivasi neural network dan optimasi gradien (Adam/SGD), sedangkan MinMaxScaler sangat sensitif dan rentan tertekan oleh satu nilai outlier ekstrem.

■ ***Poin Kunci Jawaban:*** *Z-score scaling membuat permukaan loss lebih simetris dan tidak rentan terdistorsi oleh pencilan (outliers).*

BAGIAN 2: ARSITEKTUR & TOPOLOGI NEURAL NETWORK

[Q5] KERAS API PARADIGM: Mengapa Model 1 & 2 dibangun menggunakan Sequential, sedangkan Model 3 & 4 menggunakan Functional API (Model)?

```
Potongan Kode Terkait:
# Model 1/2: model = Sequential([...])
# Model 4: inputs = layers.Input(...); x = ...; model = Model(inputs=inputs, outputs=outputs)
```

Jawaban Komprehensif:
Keras Sequential dirancang untuk arsitektur sekuensial linear murni di mana layer tersusun satu arah dari input langsung ke output tanpa percabangan. Sebaliknya, **Keras Functional API** diperlukan saat membangun arsitektur non-linear kompleks yang memiliki banyak input/output, penggunaan ulang layer, atau **percabangan jalan pintas (Skip Connections)**. Pada Model 4 (ResMLP), kita melakukan operasi penjumlahan elemen layers.Add()([x, res1]) yang menghubungkan input blok dengan output blok. Operasi *residual add* ini mustahil direpresentasikan secara murni oleh Sequential.

■ ***Poin Kunci Jawaban:*** *Functional API mutlak dibutuhkan untuk arsitektur non-sekuensial seperti residual block (skip connections).*

[Q6] BATCH NORMALIZATION & BIAS: Mengapa pada Regularized_MLP, layer Dense menggunakan use_bias=False sebelum BatchNormalization?

```
Potongan Kode Terkait:
x = layers.Dense(128, kernel_regularizer=regularizers.l2(l2_reg), use_bias=False)(inputs)
x = layers.BatchNormalization()(x)
x = layers.Activation('relu')(x)
```

Jawaban Komprehensif:
Layer BatchNormalization memiliki parameter skala γ dan parameter pergeseran (*shift/bias*) β bawaan: $y = \gamma \hat{x} + \beta$. Sebelum mengalikan dengan γ dan menambahkan β, BatchNorm menormalkan input dengan mengurangkan nilai rata-rata batch (μ_B). Jika kita mengaktifkan bias pada Dense layer ($Wx + b$), maka bias b tersebut akan langsung tereliminasi saat dikurangi oleh μ_B: $(Wx + b) - (\mu(Wx) + b) = Wx - \mu(Wx)$. Akibatnya, bias pada Dense menjadi parameter redundan (sia-sia). Menyetel use_bias=False menghemat parameter dan mempercepat komputasi tanpa mengurangi representasi model.

■ ***Poin Kunci Jawaban:*** *Bias pada Dense redundan karena terhapus oleh operasi pengurangan rata-rata pada Batch Normalization.*

[Q7] SKIP CONNECTION MATH: Apa fungsi matematis dari layers.Add()([x, res1]) pada Residual MLP? Mengapa dimensinya harus sama persis?

```
Potongan Kode Terkait:
res1 = layers.Dense(hidden_dim, activation='gelu')(x)
res1 = layers.Dropout(0.2)(res1)
res1 = layers.Dense(hidden_dim)(res1)
x = layers.Add()([x, res1])
```

Jawaban Komprehensif:
Operasi tersebut mengimplementasikan formulasi Residual Learning: $\mathcal{H}(x) = \mathcal{F}(x) + x$, di mana x adalah fitur identitas (*identity mapping*) dan $\mathcal{F}(x)$ adalah transformasi non-linear yang dipelajari residual block. Selama *backpropagation*, turunan gradien terhadap input adalah $\frac{\partial \mathcal{H}}{\partial x} = \frac{\partial \mathcal{F}}{\partial x} + 1$. Konstanta +1 ini bertindak sebagai **Gradient Highway**, menjamin aliran gradien tidak akan lenyap (*vanishing gradient*) meskipun jaringan sangat dalam. Operasi Add() melakukan penjumlahan elemen-demi-elemen (*element-wise addition*), sehingga dimensi vektor x dan $\mathcal{F}(x)$ wajib sama persis (keduanya berukuran hidden_dim=64).

■ ***Poin Kunci Jawaban:*** *Menerapkan residual learning H(x) = F(x) + x yang menjamin aliran gradien balik tanpa lenyap (gradient highway).*

[Q8] ACTIVATION FUNCTION: Mengapa pada ResMLP Anda menggunakan fungsi aktivasi GELU dan bukan ReLU?

```
Potongan Kode Terkait:
res1 = layers.Dense(hidden_dim, activation='gelu')(x)
```

Jawaban Komprehensif:

GELU (Gaussian Error Linear Unit) adalah aktivasi modern non-linear mulus yang digunakan pada model mutakhir (seperti BERT, GPT, Vision Transformer, dan modern Tabular ResNet). Berbeda dengan ReLU yang memiliki sudut patah tajam pada $x=0$ dan mematikan neuron secara permanen untuk $x < 0$ (*Dying ReLU problem*), GELU memboboti input berdasarkan probabilitas kumulatif distribusi normal: $\text{GELU}(x) = x \cdot \Phi(x)$. Ini memberikan gradien non-nol kecil untuk nilai input negatif, menghasilkan permukaan loss (*loss landscape*) yang lebih halus dan kontinuitas yang sangat cocok untuk data tabular kontinu.

■ ***Poin Kunci Jawaban:*** *GELU mencegah masalah 'Dying ReLU' dan menghasilkan loss landscape yang jauh lebih mulus dan stabil.*

[Q9] OUTPUT LAYER: Mengapa output layer menggunakan Dense(1, activation='sigmoid') dengan loss binary_crossentropy, bukan Dense(2, activation='softmax')?

```
Potongan Kode Terkait:
outputs = layers.Dense(1, activation='sigmoid')(x)
model.compile(optimizer=..., loss='binary_crossentropy')
```

Jawaban Komprehensif:

Secara matematis, untuk klasifikasi biner, Dense(1, sigmoid) dan Dense(2, softmax) ekuivalen. Namun, Dense(1, sigmoid) lebih efisien dan elegan karena hanya memerlukan 1 set bobot dan 1 bias, memotong jumlah parameter output hingga 50%. Selain itu, output skalar tunggal $\hat{y} \in [0, 1]$ secara langsung merepresentasikan probabilitas posterior pasien mengidap kanker ganas: $P(Y=1|X)$. Hal ini memudahkan dokter untuk mengubah ambang batas keputusan (*decision threshold*) jika diperlukan.

■ ***Poin Kunci Jawaban:*** *Lebih efisien (parameter 50% lebih hemat) dan menghasilkan probabilitas posterior biner skalar tunggal yang intuitif.*

BAGIAN 3: OPTIMASI, HYPERPARAMETER, & CALLBACKS

[Q10] LOSS FUNCTION FORMULATION: Bagaimana formulasi matematis dari binary_crossentropy yang dihitung di dalam kode Keras?

```
Potongan Kode Terkait:
loss='binary_crossentropy'
```

Jawaban Komprehensif:
Binary Cross-Entropy (Log Loss) mengukur jarak informasi antara distribusi label aktual $y \in \{0, 1\}$ dan probabilitas prediksi $\hat{y} \in (0, 1)$ untuk N sampel:
$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N \left[y_i \ln(\hat{y}_i) + (1 - y_i) \ln(1 - \hat{y}_i) \right]$$
Jika pasien aktual ganas ($y_i=1$) tetapi model memprediksi $\hat{y}_i \to 0$, maka $\ln(\hat{y}_i) \to -\infty$, menghasilkan penalti loss yang sangat besar. Ini memaksa model untuk memprioritaskan kalibrasi probabilitas yang akurat.

■ ***Poin Kunci Jawaban:*** *Menghitung log-loss biner yang memberikan penalti eksponensial terhadap kesalahan prediksi probabilitas.*

[Q11] DYNAMIC LEARNING RATE: Jelaskan mekanisme kerja callback ReduceLROnPlateau(factor=0.5, patience=4, min_lr=1e-5) pada script training!

```
Potongan Kode Terkait:
tf.keras.callbacks.ReduceLROnPlateau(
    monitor='val_loss', factor=0.5, patience=4, min_lr=1e-5
)
```

Jawaban Komprehensif:
Callback ini memantau metrik val_loss di setiap akhir epoch. Jika nilai val_loss tidak mengalami penurunan (perbaikan) selama 4 epoch berturut-turut (patience=4), maka laju pembelajaran (*learning rate*) saat itu akan dikalikan dengan faktor 0.5 (dipotong setengahnya): $\eta_{\text{new}} = \eta_{\text{old}} \times 0.5$. Proses pemotongan ini dibatasi hingga batas bawah min_lr=1e-5. Mekanisme ini memungkinkan model keluar dari kebuntuan (*plateau*) di dasar lembah error dan melakukan penyesuaian bobot dengan langkah yang lebih presisi.

■ ***Poin Kunci Jawaban:*** *Secara otomatis membagi dua learning rate saat validasi mengalami kebuntuan selama 4 epoch.*

[Q12] EARLY STOPPING & MODEL CHECKPOINTING: Apa fungsi parameter restore_best_weights=True pada EarlyStopping? Apa bahayanya jika disetel False?

```
Potongan Kode Terkait:
tf.keras.callbacks.EarlyStopping(
    monitor='val_loss', patience=12, restore_best_weights=True
)
```

Jawaban Komprehensif:
Ketika pelatihan dihentikan karena val_loss tidak membaik selama 12 epoch berturut-turut, model saat itu sebenarnya berada pada epoch yang sudah mulai mengalami penurunan performa (overfit). Jika restore_best_weights=False (default di versi lama), model akan mempertahankan bobot di epoch terakhir yang sudah buruk tersebut. Dengan menyetel restore_best_weights=True, Keras secara otomatis mengembalikan (*rollback*) seluruh bobot jaringan ke epoch di mana val_loss mencapai nilai terendah mutlak sepanjang sejarah pelatihan.

■ ***Poin Kunci Jawaban:*** *Mengembalikan bobot model ke performa puncak terbaik, bukan bobot di epoch terakhir yang sudah mulai overfit.*

[Q13] BATCH SIZE TRADE-OFF: Mengapa Anda menetapkan batch_size=32? Mengapa tidak menggunakan batch_size=1 atau seluruh data (455 data)?

```
Potongan Kode Terkait:
model.fit(..., batch_size=32, epochs=70)
```

Jawaban Komprehensif:
Ini adalah kompromi klasik antara *Pure SGD* ($B=1$) dan *Full-Batch Gradient Descent* ($B=455$):
1. Jika $B=1$, estimasi gradien sangat berisik (*noisy*), komputasi lambat karena tidak memanfaatkan paralelisasi GPU/vektorisasi CPU SIMD.
2. Jika $B=455$ (Full-batch), arah gradien sangat deterministik, namun komputasi cenderung mudah terjebak pada *sharp minima* lokal dan kurang memiliki sifat generalisasi yang baik.
3. Ukuran mini-batch $B=32$ memberikan sedikit derau stokastik (*stochastic noise*) yang membantu model keluar dari saddle points, sekaligus efisien secara komputasi matriks.

■ ***Poin Kunci Jawaban:*** *Mini-batch 32 menyeimbangkan keteraturan noise stokastik untuk regularisasi dan kecepatan vektorisasi CPU/GPU.*

BAGIAN 4: METRIK EVALUASI & AMBANG BATAS KLINIS

[Q14] DECISION THRESHOLD: Pada fungsi evaluasi Anda menulis y_pred = (y_prob >= 0.5).astype(int). Apakah ambang batas 0.5 ini mutlak, atau bisakah digeser?

```
Potongan Kode Terkait:
def evaluate_predictions(y_true, y_prob, threshold=0.5):
    y_pred = (y_prob >= threshold).astype(int)
```

Jawaban Komprehensif:
Ambang batas 0.5 adalah batas standar matematis, namun **sama sekali tidak mutlak dalam konteks klinis**. Dalam diagnosis kanker payudara, biaya kesalahan False Negative (kanker terlewat) jauh lebih mahal dibanding False Positive. Oleh karena itu, rumah sakit dapat menurunkan threshold ini, misalnya ke **0.3 atau 0.4**. Dengan menurunkan threshold, pasien dengan probabilitas 35% ganas akan langsung ditandai untuk pemeriksaan lanjutan, sehingga Recall (Sensitivitas) akan meningkat mendekati 100%, meskipun Presisi akan sedikit menurun.

■ ***Poin Kunci Jawaban:*** *Threshold dapat diturunkan (<0.5) di rumah sakit untuk memaksimalkan Recall demi keselamatan pasien kanker.*

[Q15] EDGE-CASE SAFETY: Apa kegunaan parameter zero_division=0 pada pemanggilan fungsi precision_score dan recall_score?

```
Potongan Kode Terkait:
prec = precision_score(y_true, y_pred, zero_division=0)
rec = recall_score(y_true, y_pred, zero_division=0)
```

Jawaban Komprehensif:
Dalam komputasi metrik klasifikasi: $\text{Precision} = \frac{TP}{TP + FP}$. Jika pada salah satu lipatan evaluasi awal model memprediksi seluruh sampel sebagai kelas negatif (0), maka $TP = 0$ dan $FP = 0$, sehingga penyebutnya menjadi 0 ($0 / 0$). Secara default, Python akan memunculkan peringatan UndefinedMetricWarning atau runtime error. Parameter zero_division=0 secara aman memberitahu scikit-learn untuk mengembalikan nilai 0.0 jika terjadi pembagian dengan nol, menjaga script eksperimen berjalan otomatis tanpa crash.

■ ***Poin Kunci Jawaban:*** *Mencegah runtime warning / ZeroDivisionError saat tidak ada prediksi positif sama sekali.*

[Q16] CLINICAL METRICS: Bagaimana kode Anda menghitung Specificity? Mengapa metrik ini penting padahal tidak disediakan otomatis di scikit-learn?

```
Potongan Kode Terkait:
tn, fp, fn, tp = confusion_matrix(y_true, y_pred).ravel()
spec = tn / (tn + fp) if (tn + fp) > 0 else 0.0
```

Jawaban Komprehensif:
Scikit-learn menyediakan recall_score yang mengukur True Positive Rate (Sensitivity), namun tidak memiliki fungsi bawaan bernama specificity_score. Specificity mengukur **True Negative Rate**: proporsi pasien sehat (tumor jinak) yang benar-benar berhasil diidentifikasi sebagai jinak: $\text{Spec} = \frac{TN}{TN + FP}$. Dalam evaluasi medis, dokter memerlukan pasangan **Sensitivitas dan Spesifisitas** secara bersamaan. Model dengan sensitivitas 100% sangat mudah dibuat (cukup mendiagnosis semua pasien sakit), tetapi model tersebut tidak berguna jika spesifisitasnya 0%. Spesifisitas tinggi membuktikan model tidak over-diagnosis.

■ ***Poin Kunci Jawaban:*** *Specificity mengukur True Negative Rate (kemampuan mengenali tumor jinak) untuk mencegah alarm palsu.*

[Q17] ROC & AUC ANALYSIS: Bagaimana fungsi roc_curve dan roc_auc_score bekerja pada output probabilitas model Anda?

```
Potongan Kode Terkait:
auc = roc_auc_score(y_test, y_test_prob)
fpr, tpr, _ = roc_curve(y_test, y_test_prob)
```

Jawaban Komprehensif:
roc_curve tidak menggunakan label biner 0/1 melainkan **probabilitas kontinu** $\hat{y} \in [0, 1]$. Fungsi ini menyortir probabilitas dan menyapu ambang batas (threshold θ) dari 1.0 ke 0.0, menghitung pasangan nilai (FPR, TPR) pada setiap ambang: $TPR(\theta) = \frac{TP(\theta)}{P}$ dan $FPR(\theta) = \frac{FP(\theta)}{N}$. roc_auc_score menghitung luas daerah di bawah kurva tersebut menggunakan aturan integrasi trapesium (*trapezoidal rule*). Nilai AUC 0.9977 membuktikan model memiliki pemisahan probabilitas yang hampir sempurna di seluruh rentang threshold.

■ ***Poin Kunci Jawaban:*** *Menyapu seluruh kemungkinan ambang batas keputusan probabilitas dan mengintegrasikan area pemisah kedua kelas.*

BAGIAN 5: K-FOLD CROSS-VALIDATION & REPRODUCIBILITY

[Q18] VARIANS MODEL K-FOLD: Mengapa evaluasi K-Fold Anda melaporkan rata-rata beserta standar deviasi (misal: 98.02% ± 0.82%)? Mengapa nilai standar deviasi ini penting?

```
Potongan Kode Terkait:
avg_metrics[k + '_mean'] = float(np.mean(vals))
avg_metrics[k + '_std'] = float(np.std(vals))
```

Jawaban Komprehensif:
Akurasi pada satu kali split bisa menipu karena faktor keberuntungan acak pembagian sampel (*sampling bias*). Dengan melaporkan **Standar Deviasi (σ)** di kelima fold, kita mengukur stabilitas dan varians model terhadap variasi data latih. Regularized MLP mencatatkan standar deviasi akurasi terendah, yaitu hanya **$\pm 0.82\%$** (dibanding Deep MLP biasa $\pm 1.28\%$). Nilai σ yang kecil membuktikan bahwa model memiliki keandalan (*reliability*) tinggi dan tidak rapuh (*brittle*) ketika diuji pada subset pasien yang berbeda.

■ ***Poin Kunci Jawaban:*** *Standar deviasi kecil membuktikan model stabil, konsisten, dan memiliki generalisasi terpercaya antar subset data.*

[Q19] OVERFITTING MITIGATION PROOF: Bagaimana kode di notebook membuktikan secara empiris bahwa teknik regularisasi berhasil mengatasi overfitting pada deep network?

```
Potongan Kode Terkait:
# Cell 6 di notebook: memplot kurva perbandingan val_loss antar model dan antar optimizer
```

Jawaban Komprehensif:
Pembuktian empiris disajikan secara langsung pada Cell 6 dan Cell 8 di notebook:
1. **Bukti Kegagalan Deep MLP Unregularized:** Pada grafik kurva pembelajaran, celah antara training loss dan validation loss melebar tajam setelah epoch 25, dan akurasi test set anjlok ke 96.49% (dengan 2 False Positive dan 2 False Negative).
2. **Bukti Keberhasilan Regularisasi:** Dengan menambahkan Batch Normalization, Dropout berjenjang (0.3, 0.2, 0.1), dan L2 regularizer pada Regularized MLP, kurva validasi bergerak beriringan dengan loss latih, dan akurasi kembali melonjak ke 98.25% dengan False Positive = 0.

■ ***Poin Kunci Jawaban:*** *Terbukti dari kurva val_loss yang rapat (tidak divergen) dan naiknya performa test set dari 96.49% ke 98.25%.*

[Q20] REPRODUCIBILITY GUARANTEE: Jika Ibu Afiahayati atau dosen penguji menjalankan kode notebook Anda di laptop beliau, apakah hasilnya dijamin persis sama? Bagaimana Anda menjaminnya?

```
Potongan Kode Terkait:
np.random.seed(42)
tf.random.set_seed(42)
train_test_split(..., random_state=42)
StratifiedKFold(..., random_state=42)
```

Jawaban Komprehensif:
Ya, hasilnya dijamin 100% deterministik dan dapat direproduksi (*fully reproducible*). Kami mengunci seluruh sumber keacakan (pseudo-random number generator) menggunakan seed konstan **42** pada:
1. NumPy generator: np.random.seed(42)
2. TensorFlow graph & layer initialization: tf.random.set_seed(42)
3. Sklearn data shuffling: train_test_split(..., random_state=42)
4. K-Fold shuffling: StratifiedKFold(..., shuffle=True, random_state=42). Siapa pun yang menjalankan notebook ini akan mendapatkan akurasi uji 98.25%, matriks kebingungan, dan metrik yang identik.

■ ***Poin Kunci Jawaban:*** *Seluruh generator angka acak (NumPy, TensorFlow, Scikit-Learn) telah dikunci dengan random seed 42 konstan.*